

DSBSC Modulation and Demodulation

Code:-

```
% DSBSC
fc = 10*10^3;
fm = 500;
Am = 1;
t = 0:10^(-6):0.01;

% Message signal
message = Am*cos(2*pi*fm*t);

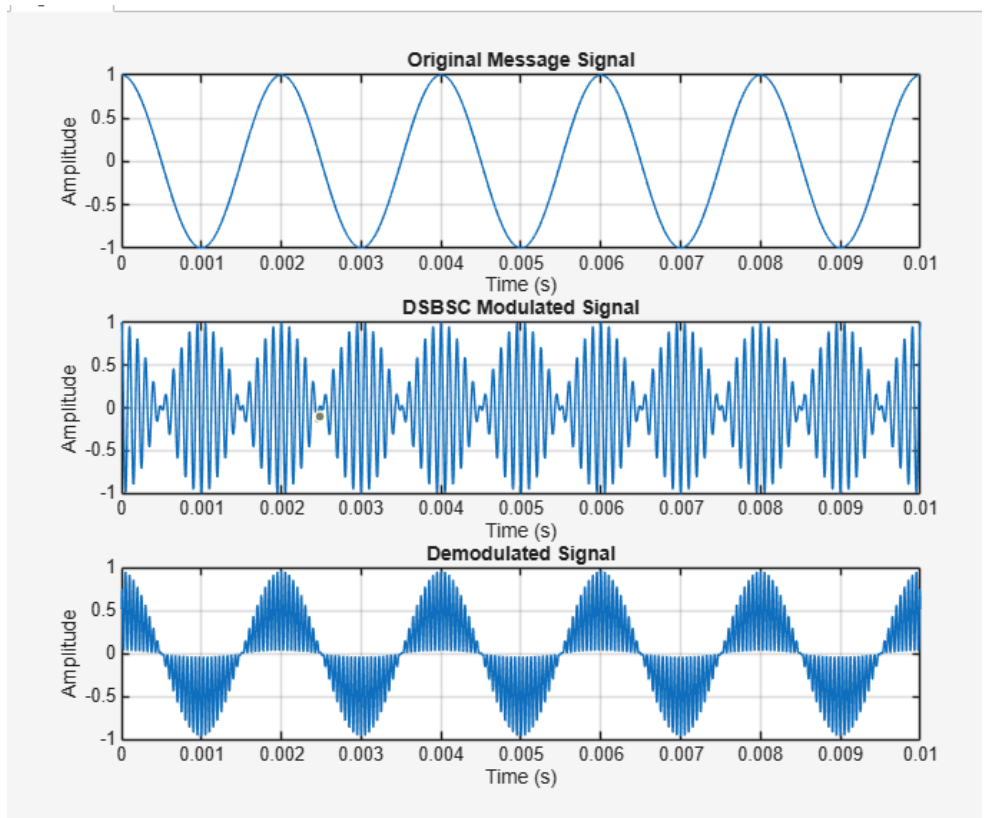
% Modulation
modulation = message .* cos(2*pi*fc*t);

% Demodulation
demodulated = modulation .* cos(2*pi*fc*t);
demodulated = lowpass(demodulated, fm, 1/(t(2)-t(1))); % Apply lowpass filter

% Plot all signals
subplot(3,1,1);
plot(t, message);
xlabel('Time (s)');
ylabel('Amplitude');
title('Original Message Signal');
grid on;

subplot(3,1,2);
plot(t, modulation);
xlabel('Time (s)');
ylabel('Amplitude');
title('DSBSC Modulated Signal');
grid on;

subplot(3,1,3);
plot(t, demodulated);
xlabel('Time (s)');
ylabel('Amplitude');
title('Demodulated Signal');
grid on;
```



SSBSC Modulation and Demodulation

Code:-

```
% SSB-SC Modulation and Demodulation
```

```
fc = 10*10^3; % Carrier frequency
```

```
fm = 500; % Message frequency
```

```
Am = 1; % Amplitude of message
```

```
t = 0:10^(-6):0.01;
```

```
% Message signal
```

```
message = Am*cos(2*pi*fm*t);
```

```
% Hilbert transform of message signal
```

```
mh = imag(hilbert(message));
```

```
% SSB-SC Modulation (Upper Sideband)
```

```
ssb = message.*cos(2*pi*fc*t) - mh.*sin(2*pi*fc*t);
```

```
% Demodulation
```

```
demodulated = ssb .* cos(2*pi*fc*t);
```

```
demodulated = lowpass(demodulated, fm, 1/(t(2)-t(1))); % Lowpass filter
```

```
% Plot signals
```

```
subplot(3,1,1);
```

```
plot(t, message);
```

```
xlabel('Time (s)');
```

```
ylabel('Amplitude');
```

```
title('Original Message Signal');
```

```
grid on;
```

```
subplot(3,1,2);
```

```
plot(t, ssb);
```

```
xlabel('Time (s)');
```

```
ylabel('Amplitude');
```

```
title('SSB-SC Modulated Signal (USB)');
```

```
grid on;
```

```
subplot(3,1,3);
```

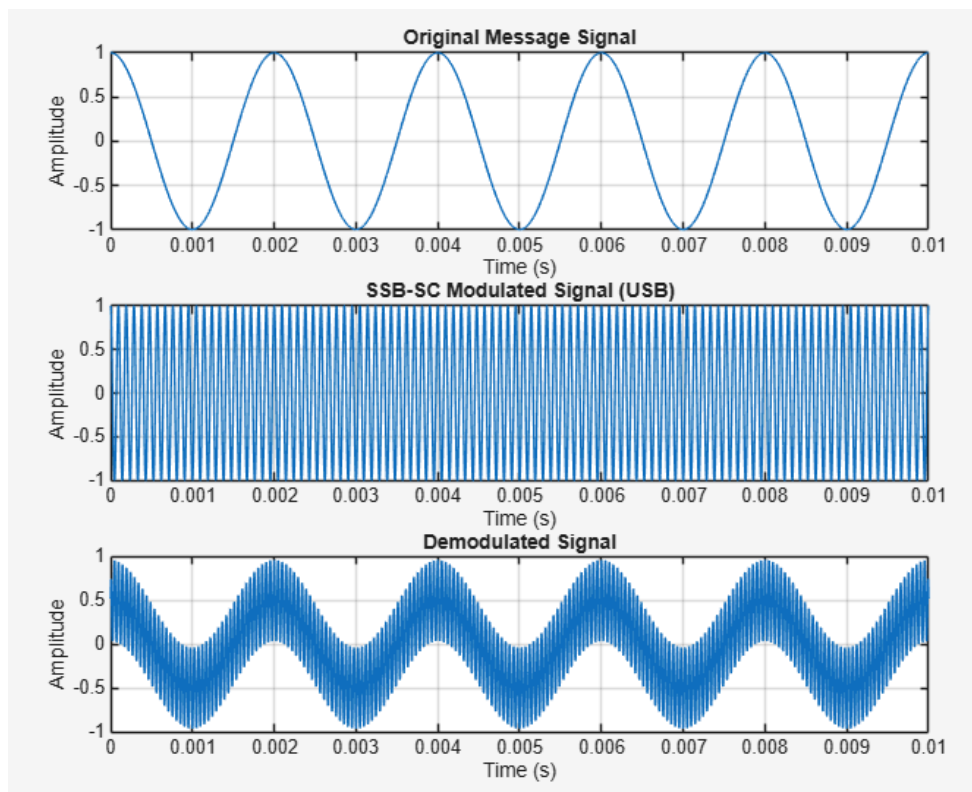
```
plot(t, demodulated);
```

```
xlabel('Time (s)');
```

```
ylabel('Amplitude');
```

```
title('Demodulated Signal');
```

```
grid on;
```



DSBFC Modulation and Demodulation

Code:-

```
% DSB-FC (AM) Modulation and Demodulation for different modulation indices
```

```
fc = 10*10^3; % Carrier frequency
fm = 500; % Message frequency
Ac = 1; % Carrier amplitude
t = 0:10^(-6):0.01;
```

```
% Message signal
message = cos(2*pi*fm*t);
```

```
% Modulation indices
mu_values = [0.5, 1, 1.5]; % Under, critical, over modulation
```

```
figure;
```

```
for i = 1:length(mu_values)
    mu = mu_values(i);
```

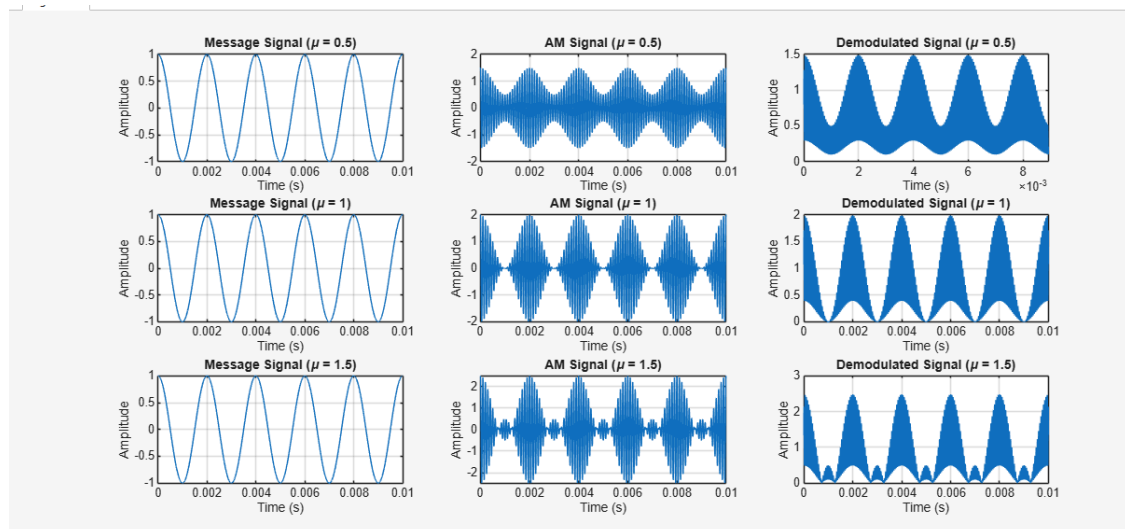
```
% AM modulation (DSB-FC)
am_signal = Ac*(1 + mu*message).*cos(2*pi*fc*t);
```

```
% Demodulation (Envelope detection using rectification + LPF)
demodulated = abs(am_signal); % Simple rectifier
demodulated = lowpass(demodulated, fm, 1/(t(2)-t(1)));
```

```
% Plotting
subplot(3,3,(i-1)*3 + 1);
plot(t, message);
title(['Message Signal (\mu = ' num2str(mu) ')']);
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```

```
subplot(3,3,(i-1)*3 + 2);
plot(t, am_signal);
title(['AM Signal (\mu = ' num2str(mu) ')']);
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```

```
subplot(3,3,(i-1)*3 + 3);
plot(t, demodulated);
title(['Demodulated Signal (\mu = ' num2str(mu) ')']);
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
end
```



PWM Modulation and Demodulation

Code:-

```
% Pulse Width Modulation and Demodulation
a = 4; % Amplitude of message signal
fs = 10000; % Sampling frequency
t = 0:(1/fs):0.1; % Time vector
f = 50; % Frequency of message signal

% Message signal (sine wave)
msg_signal = a * sin(2 * pi * f * t);

% Carrier signal (triangular wave)
fc = 500; % Carrier frequency
carrier = a * sawtooth(2 * pi * fc * t, 0.5); % Triangle wave carrier

% Pulse Width Modulation (PWM) generation
pwm = msg_signal > carrier;

% Demodulation using low-pass filter
demodulated = lowpass(double(pwm), f, fs); % Low-pass filter to extract
message

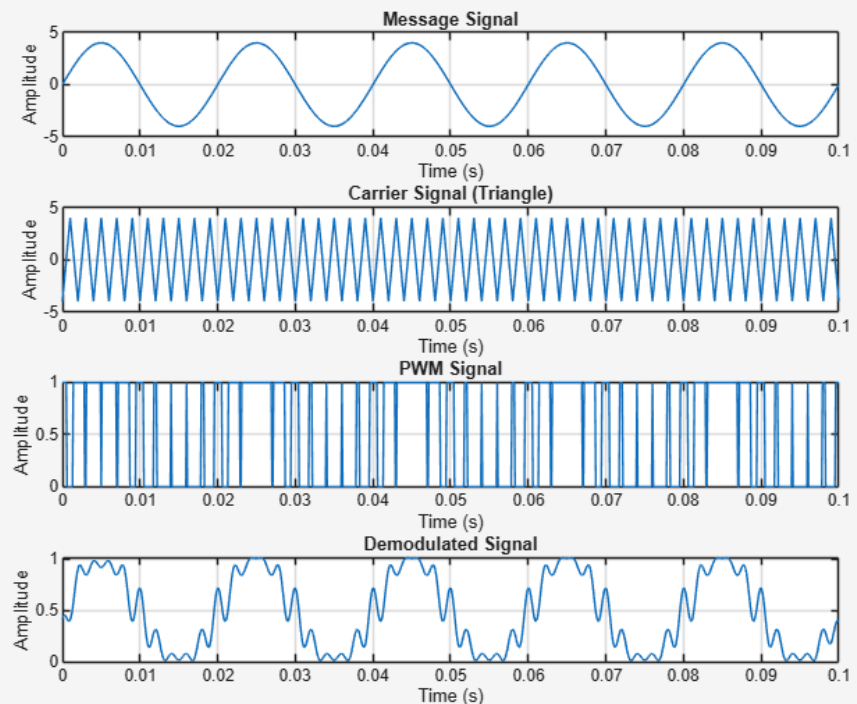
% Plotting
figure;

subplot(4,1,1);
plot(t, msg_signal);
title('Message Signal');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

subplot(4,1,2);
plot(t, carrier);
title('Carrier Signal (Triangle)');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

subplot(4,1,3);
plot(t, pwm);
title('PWM Signal');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

subplot(4,1,4);
plot(t, demodulated);
title('Demodulated Signal');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```



PPM Modulation and Demodulation

Code:-

```
clc; clear; close all;

% Parameters
fs = 10000; t = 0:1/fs:0.1;
Am = 5; fm = 50; fc = 50;
Tp = 1/fc; pulse_width = 0.001;

% Message signal and normalization
msg_signal = Am * sin(2*pi*fm*t);
msg_norm = (msg_signal + Am) / (2*Am);

% PPM signal generation
ppm = arrayfun(@(i) (mod(t(i), Tp) >= msg_norm(i)*(Tp - pulse_width)) && ...
(mod(t(i), Tp) <= msg_norm(i)*(Tp - pulse_width) + pulse_width), 1:length(t));

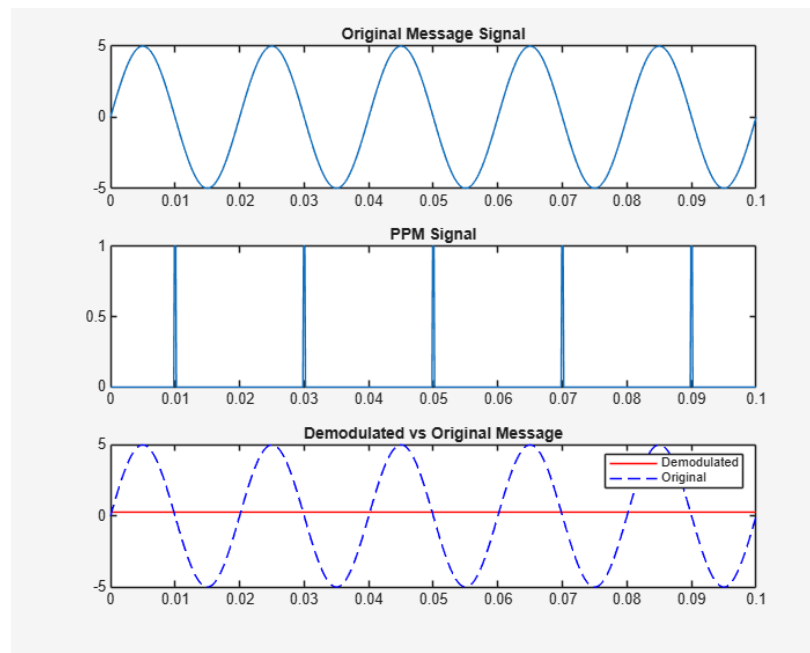
% Demodulation
n_periods = floor(max(t)/Tp);
recovered_msg_samples = zeros(1, n_periods);

for k = 0:n_periods-1
    idx_period = find(t >= k*Tp & t < (k+1)*Tp);
    idx_pulse = idx_period(ppm(idx_period) == 1);
    if ~isempty(idx_pulse)
        shift = mean(t(idx_pulse)) - k*Tp;
        recovered_msg_samples(k+1) = 2*Am*(shift/(Tp - pulse_width) - 0.5);
    end
end

% Interpolate the recovered samples
recovered_msg = interp1((0:n_periods-1)*Tp + Tp/2, recovered_msg_samples,
t, 'linear', 'extrap');

% Plotting
figure;
subplot(3,1,1); plot(t, msg_signal); title('Original Message Signal');
subplot(3,1,2); plot(t, ppm); title('PPM Signal');
subplot(3,1,3); plot(t, recovered_msg, 'r', t, msg_signal, 'b--');

% Mean Squared Error
fprintf('MSE: %.4f\n', mean((recovered_msg - msg_signal).^2));
```



PAM Modulation and Demodulation

Code:-

```
% Define parameters
fs = 1000; % Sampling frequency
f_carrier = 50; % Carrier frequency (high frequency)
f_message = 5; % Message signal frequency (low frequency)
duration = 1; % Duration in seconds
amplitude_message = 1; % Amplitude of the message signal

% Time vector
t = 0:1/fs:duration-1/fs;

% Generate message signal (e.g., a sine wave)
message_signal = amplitude_message * sin(2*pi*f_message*t);

% Generate carrier signal (a high-frequency cosine wave)
carrier_signal = cos(2*pi*f_carrier*t);

% PAM modulation: modulate the message signal by varying the amplitude of
the carrier
% Message signal's amplitude modulates the carrier amplitude
modulated_signal = carrier_signal .* message_signal;

% Plot the message signal and modulated signal
figure;

% Plot message signal
subplot(3,1,1);
plot(t, message_signal);
title('Message Signal (Baseband)');
xlabel('Time [s]');
ylabel('Amplitude');
grid on;

% Plot modulated signal (PAM)
subplot(3,1,2);
plot(t, modulated_signal, 'r');
title('PAM Modulated Signal');
xlabel('Time [s]');
ylabel('Amplitude');
grid on;

% Plot the demodulated signal
subplot(3,1,3);
plot(t, modulated_signal);
title('PAM Demodulated Signal');
xlabel('Time [s]');
ylabel('Amplitude');
grid on;
```

